

5 Categories of Tickets (for demo) :

Category	Why it is chosen
Wifi / Internet Support	Highest-volume, classic vague complaint ("internet not working"), strong KB deflection potential (reconnect steps), and the best duplicate-burst story (one AP/building outage → many near identical tickets in a short window).
VIT Email Support	Password/access issues are deflectable via KB, and it's the anchor of your auth-overlap story.
Microsoft Teams Support	You specifically wanted an "authentication-like" example - Teams login/meeting-access failures are frequently really an SSO/email-auth issue, not a Teams-specific one. This is your cleanest misclassification demo: user picks "Teams," ground truth is "VIT Email/AD."
AD Account Creation	Completes the auth-overlap triangle (Email - Teams - AD). Without this third point, "misclassification" is just two categories confusing each other, which is a weaker claim than a genuine three-way overlap cluster.
Printer Support	Different failure shape from the other four (device/hardware, not account/access) - gives you diagnostic-clarification variety (which printer, which building, error code) and strong KB deflection (driver reinstall, paper jam).

Four working days (today through Sept 1) before a Sept 2 review is tight but doable if you cut ruthlessly and parallelize from hour one. Here's the plan.

Team split (map to your existing division)

- **Person A — Intake side:** Input Understanding + Clarify agent
- **Person B — Knowledge side:** Deflect agent (KB/RAG) + Duplicate-check agent
- **Person C — Backbone:** Orchestrator (LangGraph), Classify & route agent, frontend, integration, audit logging

Dataset generation is shared work everyone does on Day 1 before splitting into their lanes — no one can build against fake data another person invented ad hoc.

Tech stack (locked for speed, not ideal-world)

Layer	Choice	Why this, given 4 days
-------	--------	------------------------

Orchestration	LangGraph	You already committed to this; nodes = your 4 functions, router = orchestrator
Backend	FastAPI	Fast to stand up, async-friendly for LLM calls
Frontend	Single-page Next.js chat UI (or plain React if faster for your team)	Don't build multi-screen polish — one scrolling chat + a ticket-preview card at the end is enough
LLM	Azure OpenAI (mini-tier model)	Already provisioned; use JSON-mode/function-calling for structured outputs (classification, extraction) instead of free text parsing
Embeddings	<code>all-MiniLM</code> or <code>bge-small</code> via sentence-transformers	Free, runs on CPU, no API cost
Vector store	ChromaDB, not Postgres+pgvector, for this demo	Zero setup time — a persisted Chroma collection is running in minutes. Defer pgvector migration to the October MVP; say so explicitly in the panel talk, don't hide it
Audit log	SQLite (not Postgres)	Same reasoning — one file, zero infra setup, still gives you a real queryable audit trail for the demo
Classification	Few-shot LLM prompting with structured JSON output	No fine-tuning is happening in 4 days — be upfront about this. Fine-tuning is explicitly an October/post-CTS-data task

This is a real scope cut from your fuller stack (Postgres/pgvector), and it's the honest one to make under this deadline — flag it as a deliberate demo-only simplification in your presentation, not a silent substitution.

Day-by-day

Day 1 — today (Aug 28 evening) + Aug 29: Foundation + synthetic data (all three, together)

- Finalize the 5 categories and each category's completeness checklist (30 min, all three)
- Split dataset generation by category: each person generates/reviews tickets for ~1.5 categories using an LLM-assisted generation script (prompt the LLM to produce tickets varying completeness / deflectable / duplicate-cluster / mislabeled-category, per the axes from before)

- Target: **~30–40 tickets per category, ~175 total** — enough to demo, not a real eval-grade set
- Hand-write 5–8 KB articles per category (do this yourselves, don't LLM-generate the KB — deflection accuracy depends on it being actually correct)
- Person C in parallel: scaffold the FastAPI project, LangGraph skeleton with 4 dummy nodes + router, Chroma and SQLite wired up empty
- End of Day 1: dataset generated (unreviewed), skeleton runs end-to-end with stub logic

Day 2 — Aug 30: Build the agents in parallel

- Person A: Input Understanding (entity/field extraction via structured LLM call) + Clarify agent (completeness scoring against the checklist, one-question generation, 2–3 turn cap with fallback)
- Person B: embed the KB into Chroma, build Deflect (retrieval + confidence threshold + resolution response) and Duplicate-check (embed the ~175 tickets, threshold-based similarity match against your pre-built duplicate clusters)
- Person C: Classify & route (few-shot structured classification across the 5 categories with confidence score), orchestrator routing logic (real thresholds now, not stubs), SQLite audit writes at every gate
- End of Day 2: all 4 nodes individually functional in isolation, not yet wired together

Day 3 — Aug 31: Integration + frontend

- Morning: wire all 4 nodes into the LangGraph graph with the real orchestrator router; run your full ticket set through it end-to-end and fix breakage (this always takes longer than expected — start it in the morning, not the afternoon)
- Afternoon: Person C builds the frontend chat flow (input → clarify Q&A if triggered → deflect offer if triggered → ticket preview); Persons A and B spend the afternoon hand-picking and rehearsing the **specific 3–4 examples** you'll actually demo (one clean Clarify case, one clean Deflect case, one duplicate-cluster case, one misclassification-correction case)
- End of Day 3: full pipeline runs live on your rehearsed examples

Day 4 — Sept 1: Harden, don't add features

- Morning: bug fixes only, no new functionality. Test your 3–4 demo examples 5+ times each for consistency (LLM outputs vary — you need to know your demo script is reliable)
- Midday: freeze the code
- Afternoon: full team rehearsal — spoken walkthrough + live demo timing, prep for the "why is duplicate-check not fully live" or "why SQLite not Postgres" question if it comes up
- Evening (EOD Sept 1): done

What to explicitly defer and say so in the panel

- Postgres/pgvector migration (Chroma/SQLite is demo-scope, stated as such)

- Fine-tuned classifier (few-shot only; fine-tuning happens once CTS data arrives)
- Real CTS integration (structured JSON output, not a live API call)
- Full 175-ticket evaluation metrics (report on your rehearsed subset now; full eval suite is a 6-month-window item per your own roadmap)

Naming these deferrals up front is more defensible than a panelist discovering them — same principle you've applied throughout this project.